

PRACTICAL WEEK 8

1. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical8.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("==== Dynamic Memory Allocation =====\n\n");

    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        a[i] = i + 1;
    }

    printf("malloc(): Allocated 5 integers\n");

    printf("Values: ");
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n\n");

    int *b = (int *)calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc(): Allocated 5 integers\n");
    printf("Initial values: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", b[i]);
    }

    printf("\n\n");

    int *temp = (int *)realloc(a, 10 * sizeof(int));

    if (temp == NULL)
    {
        printf("realloc failed\n");
        free(a);
        free(b);
    }
}
```

```

        return 1;
    }

    a = temp;

    for (int i = 5; i < 10; i++)
    {
        a[i] = i + 1;
    }

    printf("realloc(): Increased allocation from 5 to 10 integers\n");
    printf("New values: ");

    for (int i = 0; i < 10; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\n\n");

    free(a);
    printf("free(): Released malloc/realloc memory\n");

    free(b);
    printf("free(): Released calloc memory\n");

    int *leak = (int *)malloc(100 * sizeof(int));

    if (leak != NULL)
    {
        leak[0] = 999;
        printf("\nIntentional memory leak created: 100 integers\n");
    }

    printf("Program completed.\n");

    return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical8.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical8.1.c -o practical8.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical8.1

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical8.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical8.1.c -o practical8.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical8.1
===== Dynamic Memory Allocation =====

malloc(): Allocated 5 integers
Values: 1 2 3 4 5

calloc(): Allocated 5 integers
Initial values: 0 0 0 0 0

realloc(): Increased allocation from 5 to 10 integers
New values: 1 2 3 4 5 6 7 8 9 10

free(): Released malloc/realloc memory
free(): Released calloc memory

Intentional memory leak created: 100 integers
Program completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Install Valgrind by using this commands: `sudo apt update`

`sudo apt install valgrind`

and after installing check the version by: `valgrind --version`

then Run: `valgrind --leak-check=full ./practical8.1`

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ valgrind --leak-check=full ./practical8.1
==2319== Memcheck, a memory error detector
==2319== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2319== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2319== Command: ./practical8.1
==2319==
===== Dynamic Memory Allocation =====

malloc(): Allocated 5 integers
Values: 1 2 3 4 5

calloc(): Allocated 5 integers
Initial values: 0 0 0 0 0

realloc(): Increased allocation from 5 to 10 integers
New values: 1 2 3 4 5 6 7 8 9 10

free(): Released malloc/realloc memory
free(): Released calloc memory

Intentional memory leak created: 100 integers
Program completed.
==2319==
==2319== HEAP SUMMARY:
==2319==   in use at exit: 400 bytes in 1 blocks
==2319==   total heap usage: 5 allocs, 4 frees, 1,504 bytes allocated
==2319==
==2319== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==2319==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==2319==    by 0x40014B6: main (in /home/vishwa_teja_veerandi/practical8.1)
==2319==
==2319== LEAK SUMMARY:
==2319==   definitely lost: 400 bytes in 1 blocks
==2319==   indirectly lost: 0 bytes in 0 blocks
==2319==   possibly lost: 0 bytes in 0 blocks
==2319==   still reachable: 0 bytes in 0 blocks
==2319==   suppressed: 0 bytes in 0 blocks
==2319==
==2319== For lists of detected and suppressed errors, rerun with: -s
==2319== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Then go to the code and change the ending line like this:

```
int *test = malloc(100 * sizeof(int));

if (test == NULL)
{
    printf("malloc failed\n");
    return 1;
}

test[0] = 999;

printf("Extra memory allocated for leak testing\n");

free(test);
test = NULL;

printf("Extra memory freed successfully\n");

printf("\nProgram completed.\n");

return 0;
}
```

Now run the code:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical8.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc -g -O0 practical8.1.c -o practical8.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical8.1
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical8.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc -g -O0 practical8.1.c -o practical8.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical8.1
===== Dynamic Memory Allocation =====

malloc(): Allocated 5 integers
calloc(): Allocated 5 integers
realloc(): Increased allocation to 10 integers
free(): Released malloc/realloc memory
free(): Released calloc memory
Extra memory allocated for leak testing
Extra memory freed successfully

Program completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Now run: `valgrind --leak-check=full --show-leak-kinds=all ./practical8.1`

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ valgrind --leak-check=full --show-leak-kinds=all ./practical8.1
==2571== Memcheck, a memory error detector
==2571== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2571== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2571== Command: ./practical8.1
==2571==
===== Dynamic Memory Allocation =====

malloc(): Allocated 5 integers
calloc(): Allocated 5 integers
realloc(): Increased allocation to 10 integers
free(): Released malloc/realloc memory
free(): Released calloc memory
Extra memory allocated for leak testing
Extra memory freed successfully

Program completed.
==2571==
==2571== HEAP SUMMARY:
==2571==   in use at exit: 0 bytes in 0 blocks
==2571==   total heap usage: 5 allocs, 5 frees, 1,504 bytes allocated
==2571==
==2571== All heap blocks were freed -- no leaks are possible
==2571==
==2571== For lists of detected and suppressed errors, rerun with: -s
==2571== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

2. Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical8.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MEMORY_SIZE (100 * 1024 * 1024)

void print_memory_usage(const char *process_name)
{
    char filename[64];
    char line[256];

    snprintf(filename, sizeof(filename),
             "/proc/self/status");

    FILE *file = fopen(filename, "r");

    if (file == NULL)
    {
        perror("fopen");
        return;
    }

    while (fgets(line, sizeof(line), file))
    {
        if (strncmp(line, "VmRSS:", 6) == 0)
        {
            printf("%s - %s", process_name, line);
            break;
        }
    }

    fclose(file);
}

int main()
{
    printf("==== Copy-on-Write Demonstration =====\n\n");

    char *memory = malloc(MEMORY_SIZE);

    if (memory == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (size_t i = 0; i < MEMORY_SIZE; i += 4096)
    {
        memory[i] = 1;
    }

    printf("Parent PID = %d\n", getpid());

    print_memory_usage("Parent before fork");

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(memory);
    }
}
```

```

    return 1;
}

if (pid == 0)
{
    printf("\nChild PID = %d\n", getpid());

    print_memory_usage("Child immediately after fork");

    printf("Child is modifying the shared memory...\n");

    for (size_t i = 0; i < MEMORY_SIZE; i += 4096)
    {
        memory[i] = 2;
    }

    print_memory_usage("Child after modification");

    printf("Child sleeping for 5 seconds...\n");

    sleep(5);

    free(memory);

    exit(0);
}
else
{
    sleep(2);

    printf("\nParent checking memory while child has modified it...\n");

    print_memory_usage("Parent after child modification");

    waitpid(pid, NULL, 0);

    printf("\nChild process completed.\n");

    print_memory_usage("Parent after child exit");

    free(memory);
}

return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical8.2.c -o practical8.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical8.2

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical8.2.c -o practical8.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical8.2
===== Copy-on-Write Demonstration =====

Parent PID = 2854
Parent before fork - VmRSS:      104276 kB

Child PID = 2855
Child immediately after fork - VmRSS:      103400 kB
Child is modifying the shared memory...
Child after modification - VmRSS:      103400 kB
Child sleeping for 5 seconds...
^[[A
Parent checking memory while child has modified it...
Parent after child modification - VmRSS:      104340 kB

Child process completed.
Parent after child exit - VmRSS:      104340 kB
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical8.2

```

This OUTPUT is from another terminal:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ps aux | grep practical8.2
vishwa_+  2854  1.8  1.3 105172 104340 pts/0    S+   08:35   0:00 ./practic
al8.2
vishwa_+  2855  1.8  1.3 105172 103464 pts/0    S+   08:35   0:00 ./practic
al8.2
vishwa_+  2862  0.0  0.0   4264   2436 pts/2    S+   08:35   0:00 grep --co
lor=auto practical8.2
```